

R Notebook

Code ▼

Dataset: Online Retail.xlsx

Source: [Coursera] (<https://www.coursera.org/projects/perform-exploratory-data-analysis-on-retail-data-with-python>)
(<https://www.coursera.org/projects/perform-exploratory-data-analysis-on-retail-data-with-python>))

Objectives:

1. Assess sales trends over time (daily, monthly, yearly, day of week, seasons)
2. Assess sales trends by country (top-selling vs low-selling)
3. Assess sales trends by product (top-selling vs low-selling)
4. Determine the number of orders generated by month
5. Compare monthly revenue and monthly orders
6. Determine monthly average order value
7. Determine most valuable customer(s)
8. Determine product popularity (by country)
9. Provide an high-level view of sales performance

Hide

```
# Install packages if they are not already installed
# install.packages(c("readxl", "dplyr", "ggplot2", "lubridate", "#stringr"))

# Load packages
library(readxl)
library(dplyr)
library(ggplot2)
library(lubridate)
library(stringr)
library(readxl)
```

Hide

```
# Read dataset
df <- read_excel("C:/Users/dande/Downloads/Online Retail.xlsx")
```

Hide

```
# Display top rows
head(df)
```

InvoiceNo <chr>	StockCode <chr>	Description <chr>	Quantity <dbl>	InvoiceDate <S3: POSIXct>	UnitPrice <dbl>	
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00	7.65	
6 rows 1-6 of 8 columns						

Explanation: The first six rows of the dataset.

[Hide](#)

```
# Display bottom rows
tail(df)
```

InvoiceNo <chr>	StockCode <chr>	Description <chr>	Quantity <dbl>	InvoiceDate <S3: POSIXct>	UnitPrice <dbl>	
581587	23256	CHILDRENS CUTLERY SPACEBOY	4	2011-12-09 12:50:00	4.15	
581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	
581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	
581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	
581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	
581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	

6 rows | 1-6 of 8 columns

Explanation: The last six rows of the dataset.

Hide

```
# Examine dataset structure  
str(df)
```

```
tibble [541,909 × 8] (S3: tbl_df/tbl/data.frame)  
$ InvoiceNo   : chr [1:541909] "536365" "536365" "536365" "536365" ...  
$ StockCode  : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...  
$ Description: chr [1:541909] "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE METAL LANTERN" "CREAM CUPID HEARTS COAT HANGER"  
"KNITTED UNION FLAG HOT WATER BOTTLE" ...  
$ Quantity   : num [1:541909] 6 6 8 6 6 2 6 6 6 32 ...  
$ InvoiceDate: POSIXct[1:541909], format: "2010-12-01 08:26:00" "2010-12-01 08:26:00" "2010-12-01 08:26:00" ...  
$ UnitPrice  : num [1:541909] 2.55 3.39 2.75 3.39 3.39 7.65 4.25 1.85 1.85 1.69 ...  
$ CustomerID: num [1:541909] 17850 17850 17850 17850 17850 ...  
$ Country    : chr [1:541909] "United Kingdom" "United Kingdom" "United Kingdom" "United Kingdom" ...
```

Explanation The columns in the dataset & their corresponding datatypes are: - InvoiceNo: character - StockCode: character - Description: character - Quantity: numeric - InvoiceDate: datetime - UnitPrice: numeric - CustomerID: numeric - Country: character

Hide

```
# Display no. of rows and columns  
dim(df)
```

```
[1] 541909      8
```

Explanation: The dataset has 541909 rows and 8 columns.

Hide

```
# Display descriptive stats  
summary(df)
```

InvoiceNo	StockCode	Description	Quantity
Length :541909	Length :541909	Length :541909	Min. :-80995.000
N.unique : 25900	N.unique : 4070	N.unique : 4211	1st Qu.: 1.000
N.blank : 0	N.blank : 0	N.blank : 0	Median : 3.000
Min.nchar: 6	Min.nchar: 1	Min.nchar: 1	Mean : 9.552
Max.nchar: 7	Max.nchar: 12	Max.nchar: 35	3rd Qu.: 10.000
		NAs : 1454	Max. : 80995.000

InvoiceDate	UnitPrice	CustomerID	Country
Min. :2010-12-01 08:26:00	Min. :-11062.060	Min. :12346	Length :541909
1st Qu.:2011-03-28 11:34:00	1st Qu.: 1.250	1st Qu.:13953	N.unique : 38
Median :2011-07-19 17:17:00	Median : 2.080	Median :15152	N.blank : 0
Mean :2011-07-04 13:34:57	Mean : 4.611	Mean :15288	Min.nchar: 3
3rd Qu.:2011-10-19 11:27:00	3rd Qu.: 4.130	3rd Qu.:16791	Max.nchar: 20
Max. :2011-12-09 12:50:00	Max. : 38970.000	Max. :18287	
		NAs :135080	

Explanation: For columns of 'character' datatype, statistics of length, number of unique values, number of blank values, minimum character length and maximum character length are shown. Conversely, for columns of 'numeric' and 'datetime' datatype, the minimum, 1st quartile (25% mark), median, mean, 3rd quartile (75% mark) and maximum are shown.

Hide

```
# Count duplicated rows
sum(duplicated(df))
```

```
[1] 5268
```

Explanation: 5268 duplicate rows were identified in the dataset. If kept in the analysis, these will produce inflated sales performance results.

Hide

```
# Count missing values
colSums(is.na(df))
```

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	0	1454	0	0	0	135080	0

Explanation: 'Description' and 'CustomerID' columns have 1454 and 135080 missing (NA) values, respectively.

Hide

```
# Count transactions with negative quantities
sum(df$Quantity < 0)
```

```
[1] 10624
```

Hide

```
# Display some negative-quantity transactions
head(
  df[df$Quantity < 0, ],
  10
)
```

InvoiceNo <chr>	StockCode <chr>	Description <chr>	Quantity <dbl>	InvoiceDate <S3: POSIXct>	UnitPrice <dbl>
C536379	D	Discount	-1	2010-12-01 09:41:00	27.50
C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	2010-12-01 09:49:00	4.65
C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65
C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29
C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29
C536391	21980	PACK OF 12 RED RETROSPOT TISSUES	-24	2010-12-01 10:24:00	0.29
C536391	21484	CHICK GREY HOT WATER BOTTLE	-12	2010-12-01 10:24:00	3.45
C536391	22557	PLASTERS IN TIN VINTAGE PAISLEY	-12	2010-12-01 10:24:00	1.65
C536391	22553	PLASTERS IN TIN SKULLS	-24	2010-12-01 10:24:00	1.65
C536506	22960	JAM MAKING SET WITH JARS	-6	2010-12-01 12:38:00	4.25

1-10 of 10 rows | 1-6 of 8 columns

Explanation: Rows with negative quantities amounted to 10624. If kept in the analysis, these will provide unreliable results of the retail store's sales performance.

Hide

```
# Mass Data Cleaning
df_clean <- df %>%

# Remove duplicate rows
distinct() %>%

# Remove rows without a product description
filter(!is.na(Description)) %>%

# Keep only positive quantities
filter(Quantity >= 1) %>%

# Keep only positive prices
filter(UnitPrice >= 1) %>%

# Remove non-product/service descriptions
filter(
  !Description %in% c(
    "?",
    "Adjust bad debt",
    "amazon",
    "AMAZON FEE",
    "Bank Charges",
    "check",
    "CARRIAGE",
    "CRUK Commission",
    "damages",
    "DOTCOM POSTAGE",
    "Discount",
    "Manual",
    "Next Day Carriage",
    "POSTAGE",
    "SAMPLES",
    "samples"
  )
) %>%

# Remove cancelled invoices beginning with C
filter(!str_starts(InvoiceNo, "C")) %>%
```

```
# Remove adjustments beginning with A
filter(!str_starts(InvoiceNo, "A"))
```

Explanation: Data removed were: 1. Duplicate rows 2. Rows without a product description 3. Rows with -/0 quantities 4. Rows with -/0 unit prices 5. Rows unrelated to products 6. Rows for cancelled invoices 7. Rows for adjustments

Hide

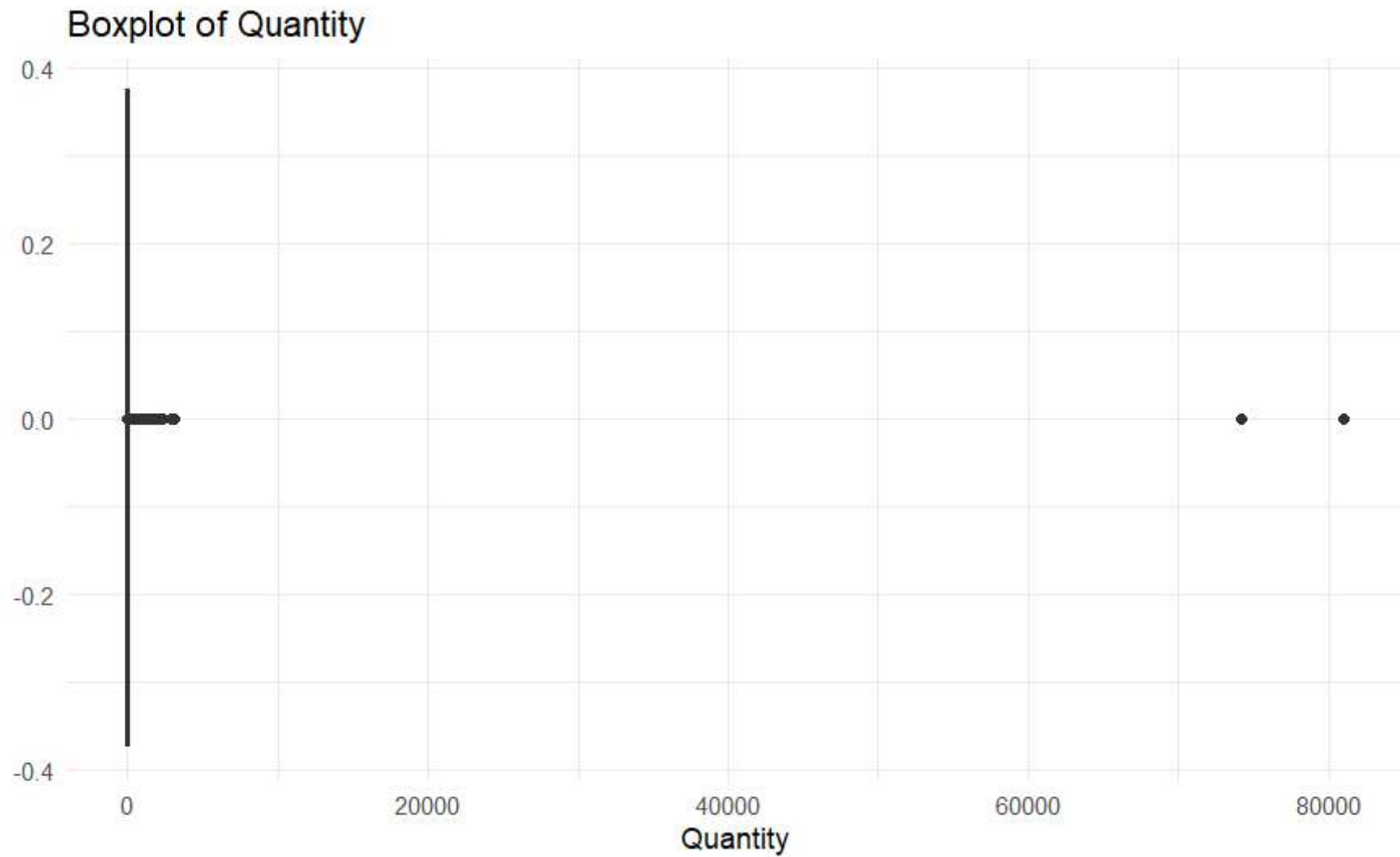
```
# View updated dataset structure
dim(df_clean)
```

```
[1] 413015      8
```

Explanation: After cleaning, 413015 rows and 8 columns are remaining.

Hide

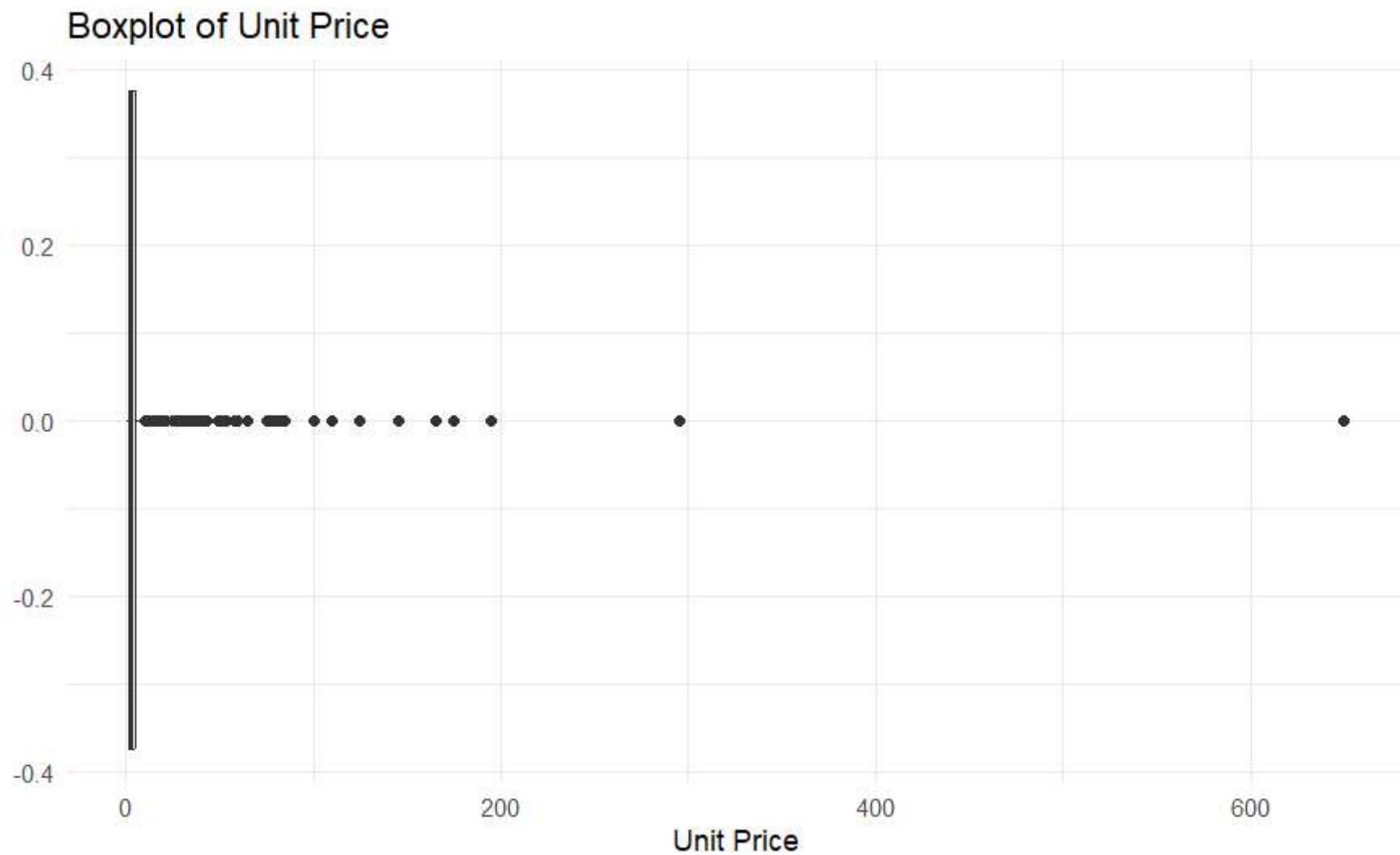
```
# Boxplot of Quantities
ggplot(df_clean, aes(x = Quantity)) +
  geom_boxplot() +
  labs(
    title = "Boxplot of Quantity",
    x = "Quantity"
  ) +
  theme_minimal()
```



Explanation: The boxplot of 'Quantity' column showed 2 quantities deviated greatly from normal. These if kept in the analysis will also provide unreliable results.

Hide

```
# Boxplot of Unit Prices
ggplot(df_clean, aes(x = UnitPrice)) +
  geom_boxplot() +
  labs(
    title = "Boxplot of Unit Price",
    x = "Unit Price"
  ) +
  theme_minimal()
```



Explanation: The boxplot of 'Unit Price' column proved that 2 unit prices deviated greatly from normal. These if kept in the analysis will also provide unreliable results.

Hide

```
# Find Quartiles of Quantity
Q1_quantity <- quantile(df_clean$Quantity, 0.25, na.rm = TRUE)
Q3_quantity <- quantile(df_clean$Quantity, 0.75, na.rm = TRUE)

IQR_quantity <- Q3_quantity - Q1_quantity

IQR_quantity
```

```
75%
7
```

Explanation: The interquartile range for 'Quantity' of 7 proves that there is a small spread of the middle 50% of the data.

Hide

```
# Find Quartiles of Unit Price
Q1_price <- quantile(df_clean$UnitPrice, 0.25, na.rm = TRUE)
Q3_price <- quantile(df_clean$UnitPrice, 0.75, na.rm = TRUE)

IQR_price <- Q3_price - Q1_price

IQR_price
```

```
75%
3.3
```

Explanation: The interquartile range for 'Unit Price' of 3.3 proves that there is a very small spread of the middle 50% of the data.

Hide

```
# Calculate lower and upper limits for Quantity
quantity_lower <- Q1_quantity - 1.5 * IQR_quantity
quantity_upper <- Q3_quantity + 1.5 * IQR_quantity

# Calculate lower and upper limits for UnitPrice
price_lower <- Q1_price - 1.5 * IQR_price
price_upper <- Q3_price + 1.5 * IQR_price

# Remove observations outside the limits
df_clean <- df_clean %>%
  filter(
    Quantity >= quantity_lower,
    Quantity <= quantity_upper,
    UnitPrice >= price_lower,
    UnitPrice <= price_upper
  )
```

Explanation: Rows outside of the interquartile boundaries for 'Quantity' and 'Unit Price' were excluded to avoid the extreme values from influencing the analysis.

Hide

```
# Display updated dataset structure
dim(df_clean)
```

```
[1] 356207      8
```

Explanation: After dropping the rows with extreme values, 356207 rows and 8 columns are remaining.

Hide

```
# Count unique countries
n_distinct(df_clean$Country)
```

```
[1] 38
```

Explanation: 38 different countries were identified in the dataset.

[Hide](#)

```
# Display all unique products
unique_products <- df_clean %>%
  select(StockCode, Description) %>%
  distinct() %>%
  arrange(Description)

# Display the unique products
unique_products
```

StockCode	Description
<chr>	<chr>
20954	*USB Office Mirror Ball
22418	10 COLOUR SPACEBOY PEN
22436	12 COLOURED PARTY BALLOONS
21448	12 DAISY PEGS IN WOOD BOX
23442	12 HANGING EGGS HAND PAINTED
21447	12 IVORY ROSE PEG PLACE SETTINGS
22906	12 MESSAGE CARDS WITH ENVELOPES
20973	12 PENCIL SMALL TUBE WOODLAND
20975	12 PENCILS SMALL TUBE RED RETROSPOT
20974	12 PENCILS SMALL TUBE SKULL
1-10 of 3,532 rows	
Previous 1 2 3 4 5 6 ... 100 Next	

[Hide](#)

```
# Calculate revenue for each transaction
df_clean$Sales <- df_clean$Quantity * df_clean$UnitPrice

# Display the first few records
head(df_clean[, c(
  "InvoiceNo",
  "Description",
  "Quantity",
  "UnitPrice",
  "Sales"
)])
```

InvoiceNo <chr>	Description <chr>	Quantity <dbl>	UnitPrice <dbl>	Sales <dbl>
536365	WHITE HANGING HEART T-LIGHT HOLDER	6	2.55	15.30
536365	WHITE METAL LANTERN	6	3.39	20.34
536365	CREAM CUPID HEARTS COAT HANGER	8	2.75	22.00
536365	KNITTED UNION FLAG HOT WATER BOTTLE	6	3.39	20.34
536365	RED WOOLLY HOTTIE WHITE HEART.	6	3.39	20.34
536365	SET 7 BABUSHKA NESTING BOXES	2	7.65	15.30
6 rows				

Explanation: The revenue, 'Quantity' x 'UnitPrice', was calculated to provide the retail store's management a sound basis for determining sales performance besides number of products sold.

Hide

```
# Convert InvoiceDate to Date format
df_clean$Date <- as.Date(
  df_clean$InvoiceDate
)

# Display the first few dates
head(df_clean$Date)
```

```
[1] "2010-12-01" "2010-12-01" "2010-12-01" "2010-12-01" "2010-12-01" "2010-12-01"
```

Explanation: The 'InvoiceDate' column has both date and time values. Only the date aspect is needed for further analysis to see how sales changes over time.

Hide

```
# Find the earliest transaction date
min(df_clean$Date)
```

```
[1] "2010-12-01"
```

Hide

```
# Find the latest transaction date
max(df_clean$Date)
```

```
[1] "2011-12-09"
```

Explanation: The earliest and latest dates in the dataset are examined to become familiar with the date range of the analysis and for better understanding of sales growth and/or decline patterns.

Hide

```
# Calculate daily sales
daily_sales <- df_clean %>%
  group_by(Date) %>%
  summarise(
    Total_Sales = sum(Sales, na.rm = TRUE)
  )

# Display the first records
head(daily_sales)
```

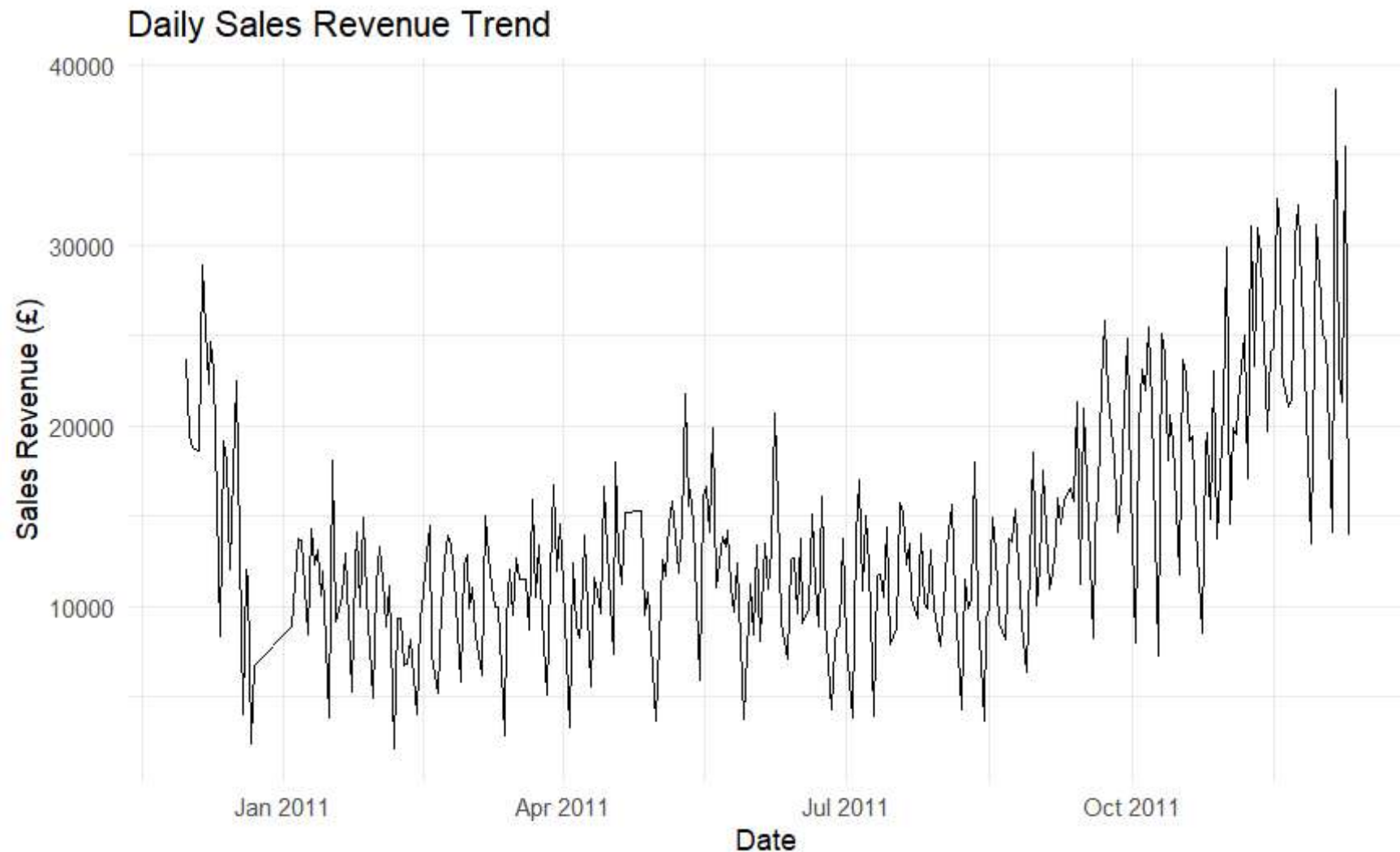
	Date <date>	Total_Sales <dbl>
	2010-12-01	23716.09
	2010-12-02	19525.48
	2010-12-03	18821.40
	2010-12-05	18679.71
	2010-12-06	28908.30
	2010-12-07	25392.03

6 rows

Explanation: Through assessing daily sales, it be an indication to the retail store's owners of the typical day(s) vs non-typical days for purchases.

Hide

```
# Display daily sales
ggplot(
  daily_sales,
  aes(
    x = Date,
    y = Total_Sales
  )
) +
  geom_line() +
  labs(
    title = "Daily Sales Revenue Trend",
    x = "Date",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanation: The daily sales revenue trend over 2010-2011 show primarily sharp rises and falls in sales. There is a staggered fall in sales in late 2010 and a gradual yet staggered increase in sales in late 2011 (roughly Nov to Dec). For the retail store, these rises and falls could be due to rapidly changing customer demands, attractive and/or unattractive sales promotions, inventory shortages or operational disruptions (e.g. website/payment system downtime, delivery delays).

Hide

```
# Calculate monthly sales
df_clean$YearMonth <- format(
  df_clean$Date,
  "%Y-%m"
)

monthly_sales <- df_clean %>%
  group_by(YearMonth) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  )

head(monthly_sales)
```

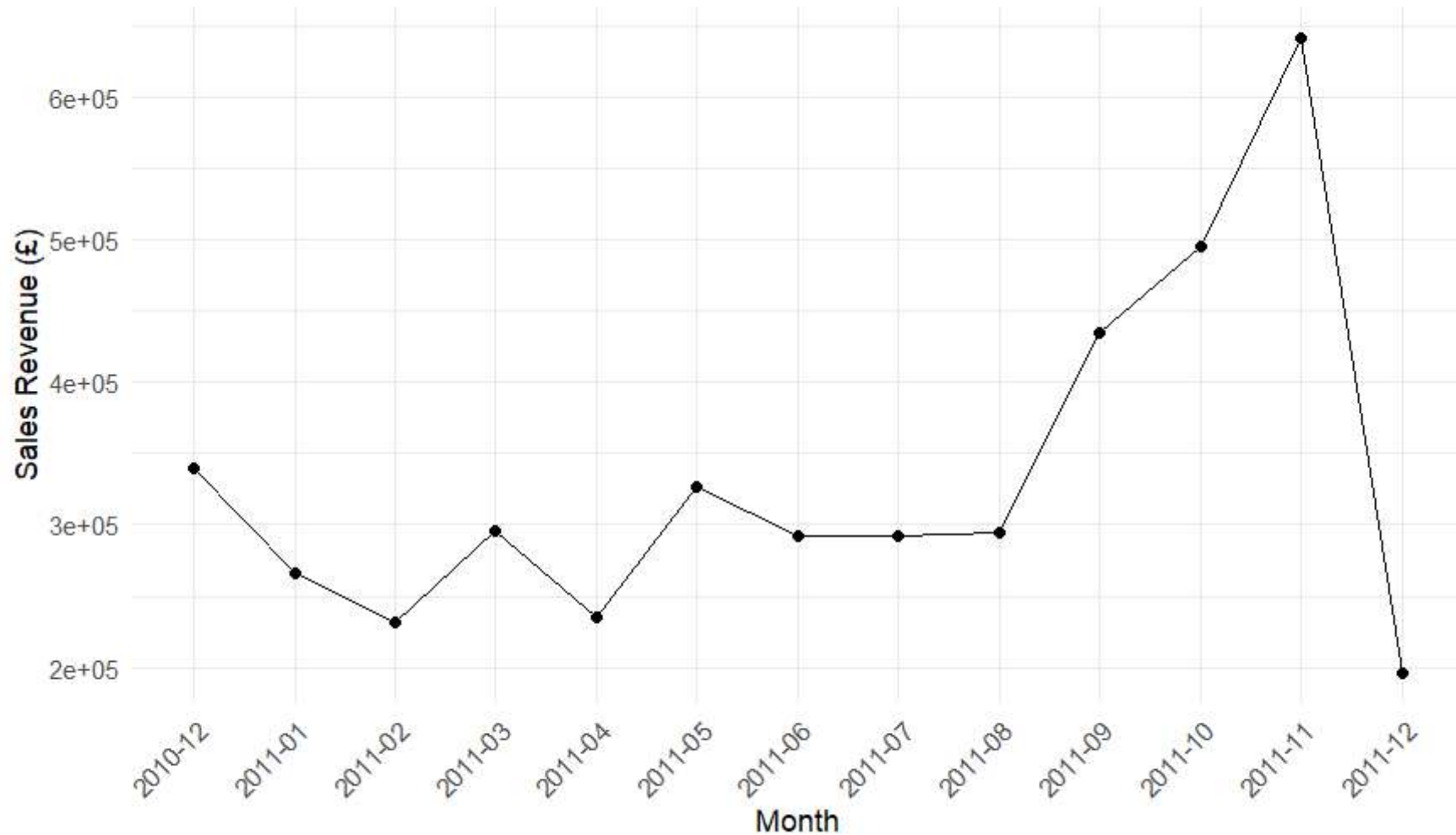
YearMonth <chr>	Total_Sales <dbl>
2010-12	339693.4
2011-01	266619.6
2011-02	231309.4
2011-03	295317.4
2011-04	234846.6
2011-05	326801.7
6 rows	

Explanation: Through assessing monthly sales, the retail store's owners can use this data for more strategic planning (eg. increasing inventory given repeated demand) and to be informed of sales influenced by seasons (eg. shopping for national holidays like Christmas).

Hide

```
# Plot monthly sales
ggplot(
  monthly_sales,
  aes(
    x = YearMonth,
    y = Total_Sales,
    group = 1
  )
) +
  geom_line() +
  geom_point() +
  labs(
    title = "Monthly Sales Revenue Trend",
    x = "Month",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
)
```

Monthly Sales Revenue Trend



Explanation: The monthly sales revenue trend for the retail store shows that it had lower sales in Dec 2010 to Aug 2011. This pattern could be attributed to weak customer demand and/or promotion by the retail store. Its sales increased after Aug 2011 to Nov 2011 then declined dramatically again in Dec 2011. This is an expected demand pattern nearing holiday seasons except for the decline in Dec 2011 which needs more context to explain.

Hide

```
# Identify the best sales months
top_months <- monthly_sales[
  order(
    monthly_sales$Total_Sales,
    decreasing = TRUE
  ),
]

head(top_months, 10)
```

YearMonth <chr>	Total_Sales <dbl>
2011-11	641792.3
2011-10	494707.4
2011-09	434966.2
2010-12	339693.4
2011-05	326801.7
2011-03	295317.4
2011-08	294970.4
2011-06	292457.5
2011-07	291629.7
2011-01	266619.6
1-10 of 10 rows	

Explanation: The best sales months were primarily in the 3rd and 4th quarters of the years. Though, not certain, this could be due to greater customer demand (local and international) given additional promotions, new product launches or changes to customer buying patterns.

Hide

```
# Identify the weakest sales months
bottom_months <- monthly_sales[
  order(
    monthly_sales$Total_Sales,
    decreasing = FALSE
  ),
]

head(bottom_months, 10)
```

YearMonth <chr>	Total_Sales <dbl>
2011-12	196308.7
2011-02	231309.4
2011-04	234846.6
2011-01	266619.6
2011-07	291629.7
2011-06	292457.5
2011-08	294970.4
2011-03	295317.4
2011-05	326801.7
2010-12	339693.4
1-10 of 10 rows	

Explanation: The weakest sales months were mainly in the 2nd and 3rd quarters of the years. This finding could be further compared to inventory levels, product options, promotion efforts, operational problems (if any) and customer buying patterns at that time for greater context.

Hide

```
# Calculate sales by year
df_clean$Year <- format(
  df_clean$Date,
  "%Y"
)

yearly_sales <- df_clean %>%
  group_by(Year) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  )

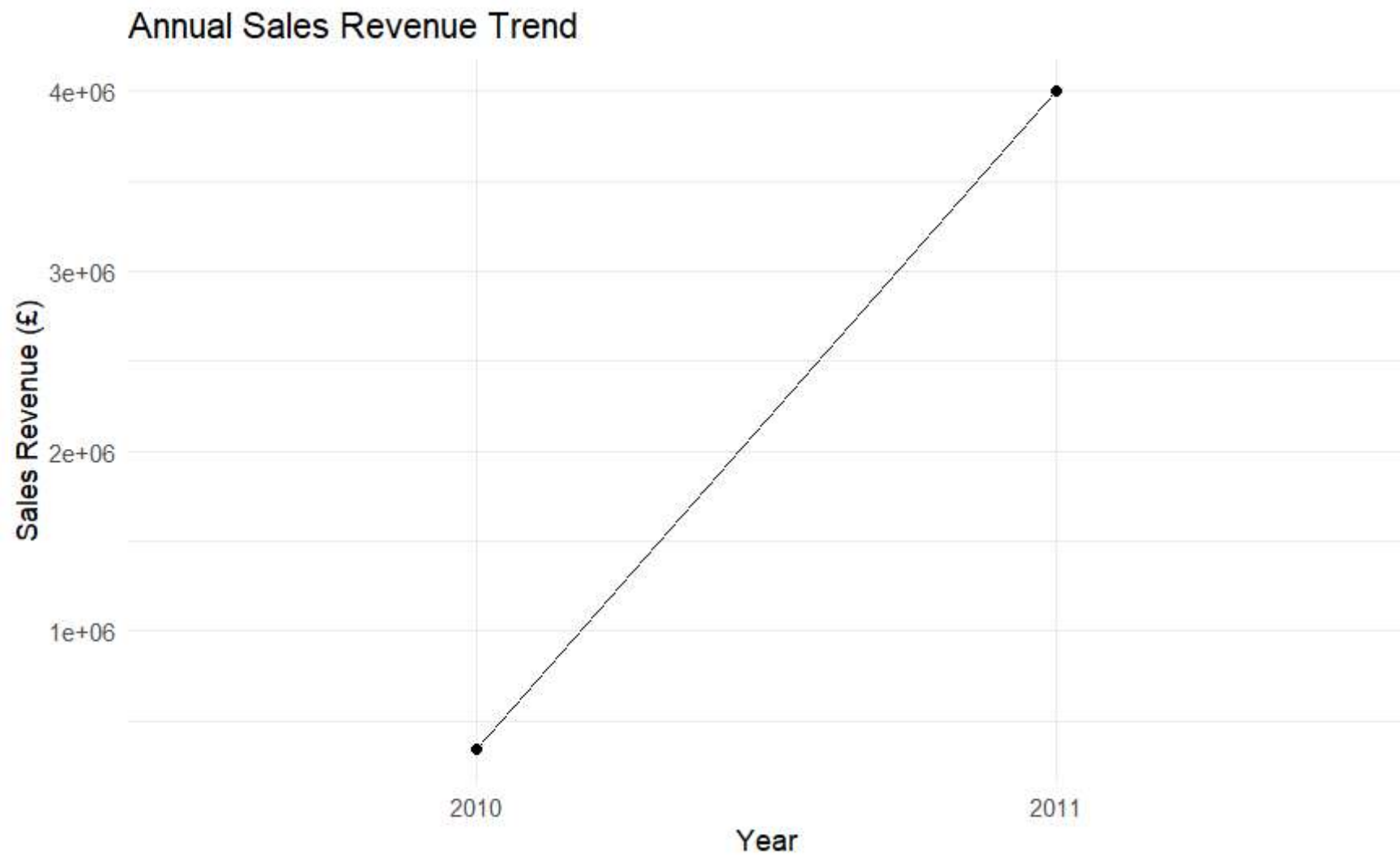
yearly_sales
```

Year	Total_Sales
<chr>	<dbl>
2010	339693.4
2011	4001726.9
2 rows	

Explanation: This yearly sales data provides the retail store's owners with a high level view of revenue. However, as 2010 only has one month represented in the dataset, this imbalance gives favour to higher sales in 2011. Consequently, the results are unsuitable for determining true demand for either years.

Hide

```
# Plot yearly sales
ggplot(
  yearly_sales,
  aes(
    x = Year,
    y = Total_Sales,
    group = 1
  )
) +
  geom_line() +
  geom_point() +
  labs(
    title = "Annual Sales Revenue Trend",
    x = "Year",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanation: The data shows an increase in yearly sales from 2010 to 2011 for the retail store; almost 12 times that of 2010. Yet, this is fostered by a dataset imbalance.

Hide

```
# Analyze sales by day of the week
df_clean$DayOfWeek <- weekdays(
  df_clean$Date
)

day_sales <- df_clean %>%
  group_by(DayOfWeek) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  )

day_sales
```

DayOfWeek	Total_Sales
<chr>	<dbl>
Friday	699910.0
Monday	733881.4
Sunday	447203.0
Thursday	880819.5
Tuesday	817319.3
Wednesday	762287.1
6 rows	

Explanation: An analysis of the sales per day of the week helps the retail store's owners to identify the weekday(s) that see the most versus least revenue. The store's owners may then dynamically adjust staffing and efficiency to fulfill orders, timing of promotions and advertising and inventory reorder schedules.

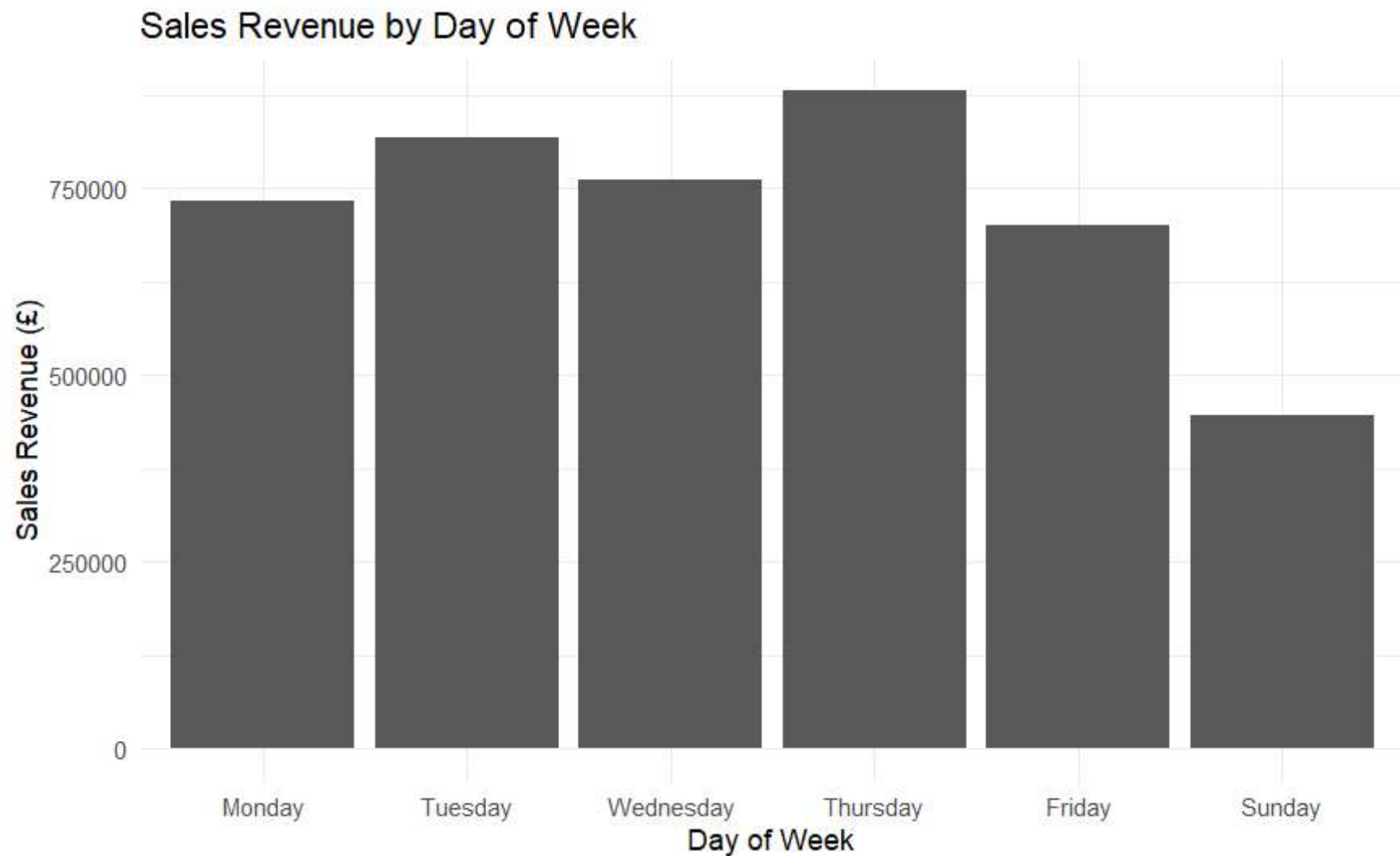
Hide

```
# Put weekdays in the correct order
day_sales$DayOfWeek <- factor(
  day_sales$DayOfWeek,
  levels = c(
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
  )
)

day_sales <- day_sales[
  order(day_sales$DayOfWeek),
]
```

Hide

```
# Plot sales by day of week
ggplot(
  day_sales,
  aes(
    x = DayOfWeek,
    y = Total_Sales
  )
) +
  geom_bar(
    stat = "identity"
  ) +
  labs(
    title = "Sales Revenue by Day of Week",
    x = "Day of Week",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanation: Tuesdays and Thursdays reflect similar revenue and the highest revenues. Mondays, Wednesdays and Fridays also had similar but lower revenue with Sunday having the lowest revenue. The retail store's owners may apply any or some of the strategies as previously suggested to capitalize on the "high revenue" days or boost sales on "low revenue" days.

Hide

```
# Analyze monthly seasonality
df_clean$Month <- format(
  df_clean$Date,
  "%m"
)

monthly_seasonality <- df_clean %>%
  group_by(Month) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  )

monthly_seasonality
```

Month <chr>	Total_Sales <dbl>
01	266619.6
02	231309.4
03	295317.4
04	234846.6
05	326801.7
06	292457.5
07	291629.7
08	294970.4
09	434966.2
10	494707.4
1-10 of 12 rows	
Previous 1 2 Next	

Explanation: Monthly seasonality helps the retail store's owners identify which month(s) overall had the best sales, specifically seasonality patterns. Therefore, the owners could implement strategies including acquiring adequate stock for the season, increase advertising/marketing efforts and optimizing order fulfillment operations ahead of these peak seasons.

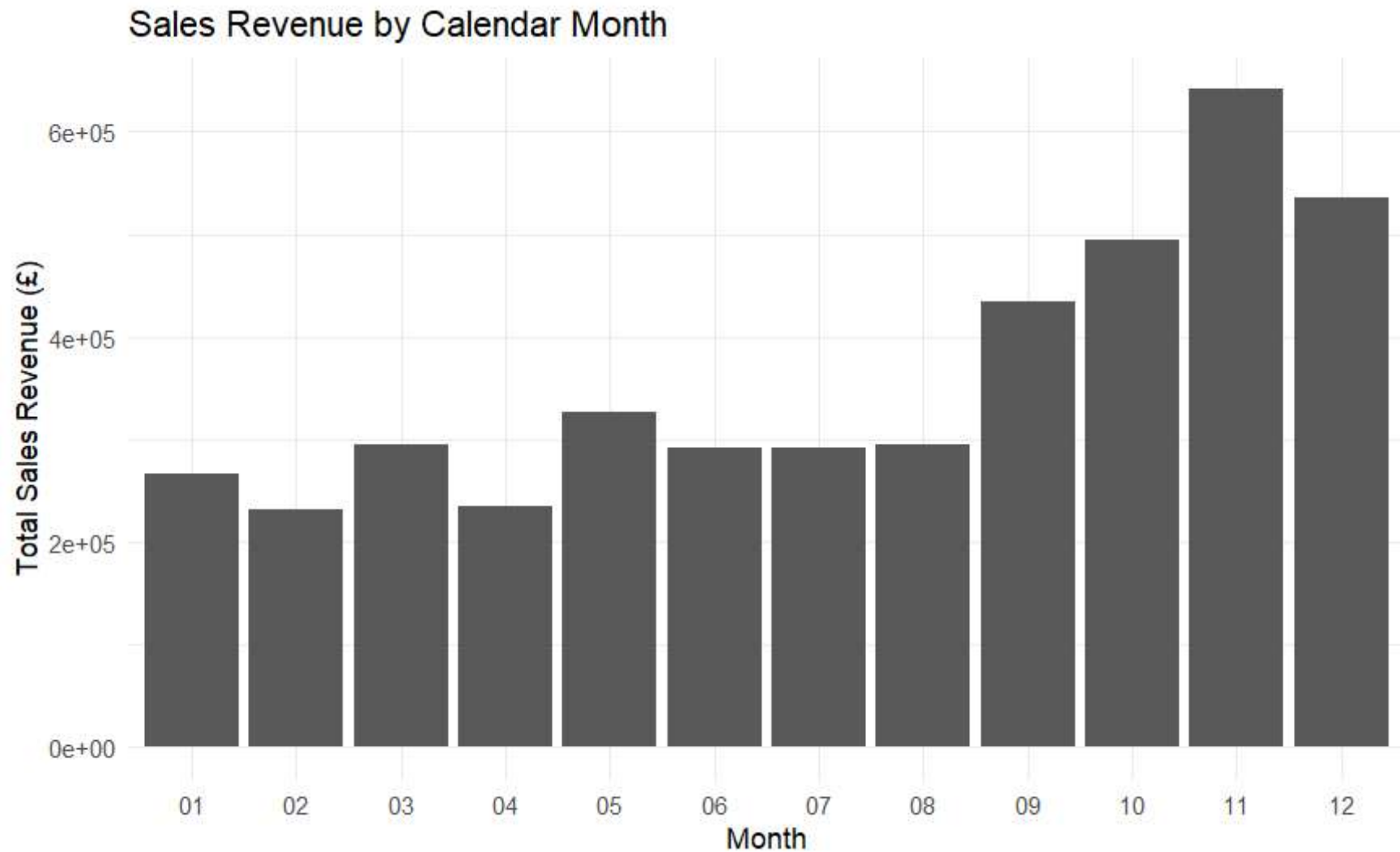
Hide

```
# Order the months correctly
monthly_seasonality$Month <- factor(
  monthly_seasonality$Month,
  levels = c(
    "01", "02", "03", "04",
    "05", "06", "07", "08",
    "09", "10", "11", "12"
  )
)

monthly_seasonality <- monthly_seasonality[
  order(monthly_seasonality$Month),
]
```

Hide

```
# Plot monthly seasonality
ggplot(
  monthly_seasonality,
  aes(
    x = Month,
    y = Total_Sales
  )
) +
  geom_bar(
    stat = "identity"
  ) +
  labs(
    title = "Sales Revenue by Calendar Month",
    x = "Month",
    y = "Total Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanation: Considering sales growth from Sept to Dec, with Nov having the highest sales, this is an indication of seasonality. These levels could be attributed to increased customer buying for holidays like Thanksgiving and Christmas celebrated during that time.

Hide

```
# Analyze international sales
country_sales <- df_clean %>%
  group_by(Country) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  ) %>%
  arrange(
    desc(Total_Sales)
  )

head(country_sales, 10)
```

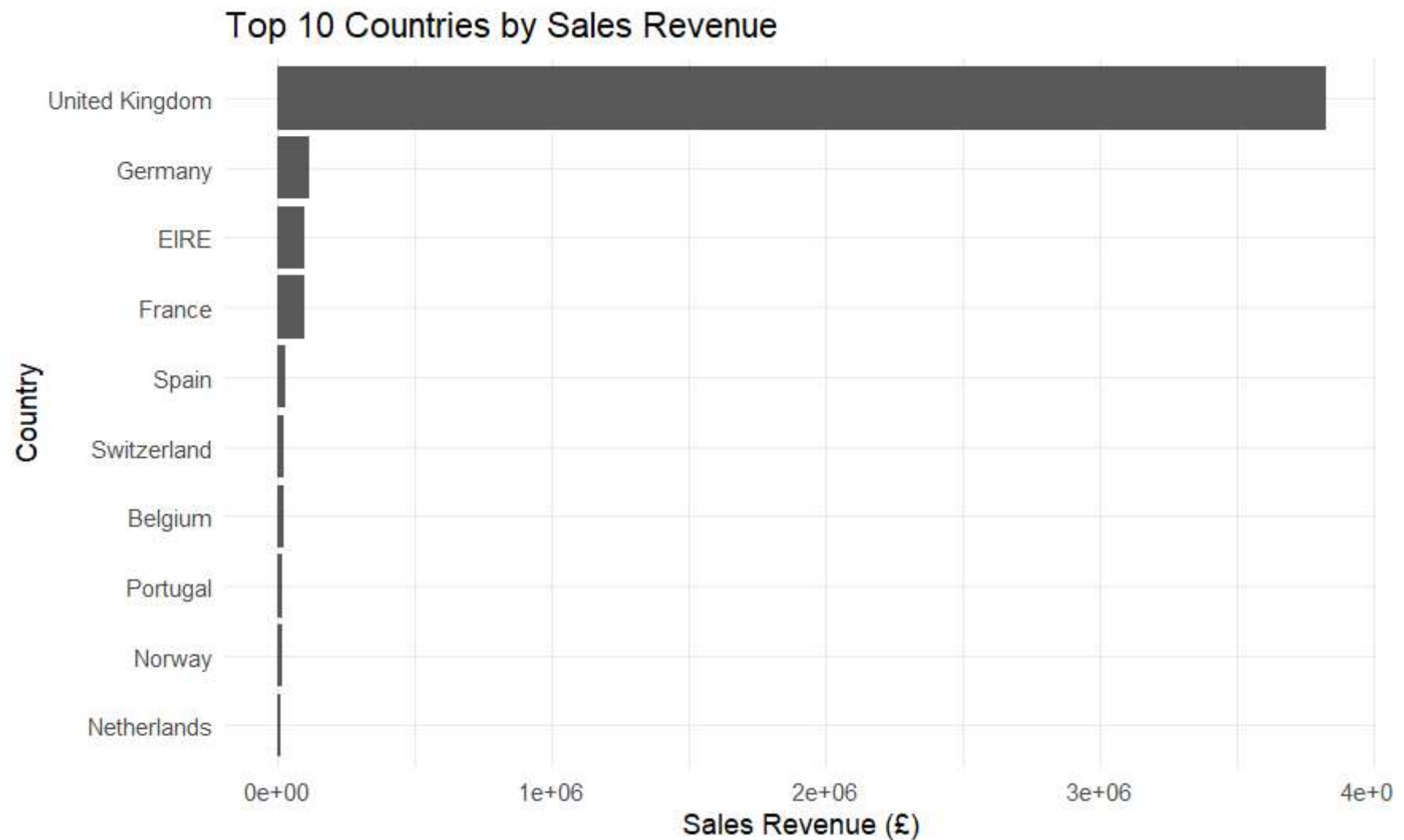
Country <chr>	Total_Sales <dbl>
United Kingdom	3824428.75
Germany	112048.66
EIRE	97781.80
France	95621.28
Spain	25233.63
Switzerland	24192.41
Belgium	24170.98
Portugal	16111.71
Norway	14674.99
Netherlands	11446.01
1-10 of 10 rows	

Explanation: Most of the online retail store's sales are generated from its base country, the United Kingdom. This may be in its owners' best favour as their marketing and shipping efforts can be minimal. However, this also presents a grand opportunity for greater international, country-specific marketing and improving shipping operations, therefore catering to a wider customer base.

Hide

```
# Plot the top 10 countries
top_countries <- head(
  country_sales,
  10
)

ggplot(
  top_countries,
  aes(
    x = reorder(
      Country,
      Total_Sales
    ),
    y = Total_Sales
  )
) +
  geom_bar(
    stat = "identity"
  ) +
  coord_flip() +
  labs(
    title = "Top 10 Countries by Sales Revenue",
    x = "Country",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanation: The United Kingdom's revenue is the millions while its runner ups' are in the hundreds of thousands. These countries (Netherlands, EIRE, Germany, France) are however good targets for further marketing efforts by the retail store to drive their revenue close to or matching that of the United Kingdom.

Hide

```
# Find the highest selling products
product_sales <- df_clean %>%
  group_by(
    StockCode,
    Description
  ) %>%
  summarise(
    Total_Sales = sum(
      Sales,
      na.rm = TRUE
    )
  ) %>%
  arrange(
    desc(Total_Sales)
  )

head(product_sales, 10)
```

StockCode <chr>	Description <chr>	Total_Sales <dbl>
47566	PARTY BUNTING	33175.65
85123A	WHITE HANGING HEART T-LIGHT HOLDER	32139.11
85099B	JUMBO BAG RED RETROSPOT	24142.42
22960	JAM MAKING SET WITH JARS	23919.30
23298	SPOTTY BUNTING	22060.05
22457	NATURAL SLATE HEART CHALKBOARD	20110.41
84879	ASSORTED COLOUR BIRD ORNAMENT	18952.95
22086	PAPER CHAIN KIT 50'S CHRISTMAS	18687.07
22726	ALARM CLOCK BAKELIKE GREEN	18599.16
22727	ALARM CLOCK BAKELIKE RED	18591.17

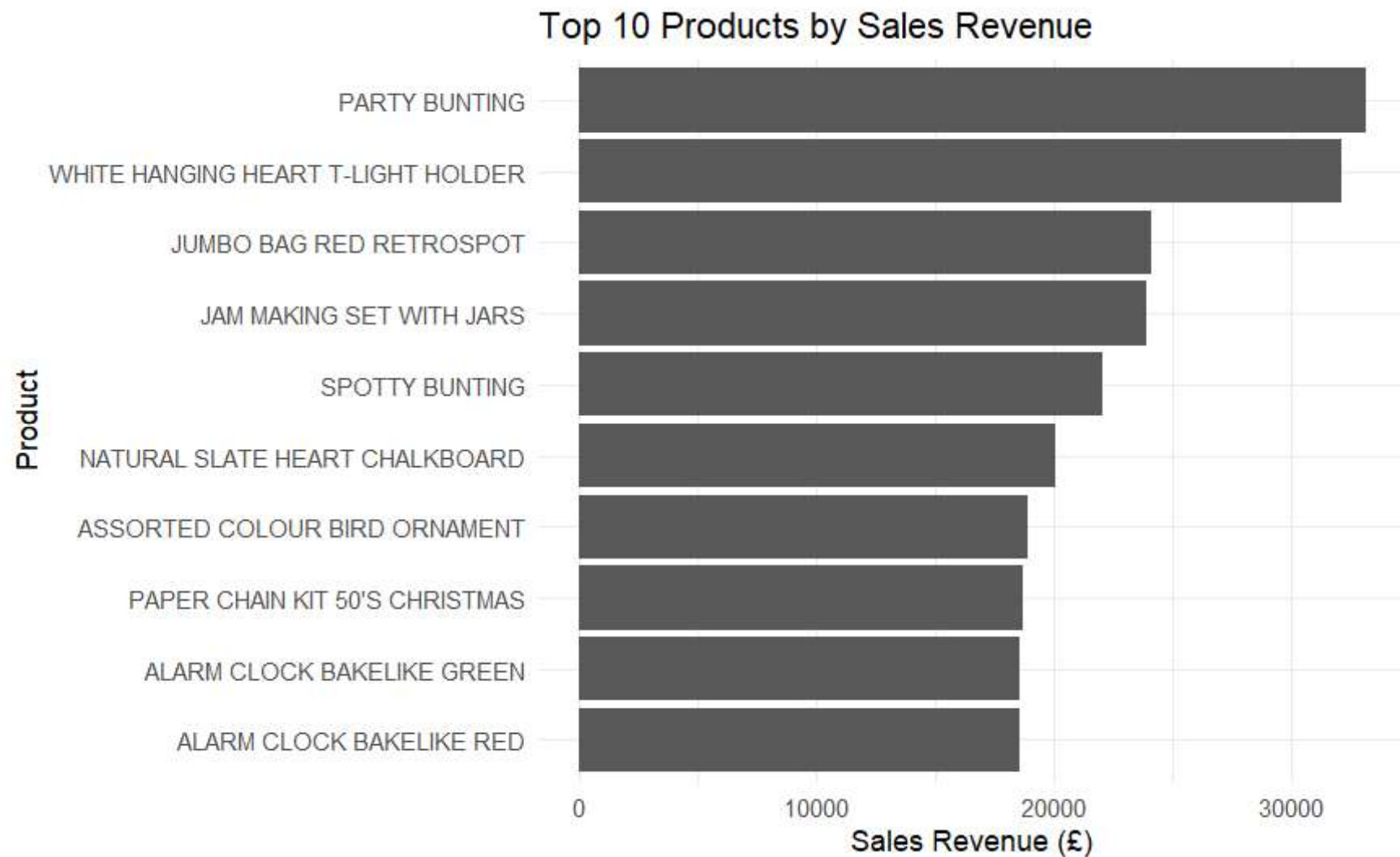
1-10 of 10 rows

Explanation: An analysis of the overall revenue per product can aid the retail store's owners in identifying the popular products, focusing inventory restock efforts, further promoting the "high sales" products and investigating the "low sales" products.

Hide

```
# Plot the top 10 products
top_products <- head(
  product_sales,
  10
)

ggplot(
  top_products,
  aes(
    x = reorder(
      Description,
      Total_Sales
    ),
    y = Total_Sales
  )
) +
  geom_bar(
    stat = "identity"
  ) +
  coord_flip() +
  labs(
    title = "Top 10 Products by Sales Revenue",
    x = "Product",
    y = "Sales Revenue (£)"
  ) +
  theme_minimal()
```



Explanations: The retail store owner's sales efforts across international markets could be centred around these 10 products to capitalize on their high revenue potential.

Hide

```
# Calculate total sales for each product within each country
country_product_sales <- df_clean %>%
  group_by(Country, StockCode, Description) %>%
  summarise(
    Total_Sales = sum(Sales, na.rm = TRUE),
    .groups = "drop"
  )

# View the results
head(country_product_sales)
```

Country <chr>	StockCode <chr>	Description <chr>	Total_Sales <dbl>
Australia	15056BL	EDWARDIAN PARASOL BLACK	17.85
Australia	20665	RED RETROSPOT PURSE	17.70
Australia	20685	DOORMAT RED RETROSPOT	79.50
Australia	20712	JUMBO BAG WOODLAND ANIMALS	61.10
Australia	20713	JUMBO BAG OWLS	19.50
Australia	20717	STRAWBERRY SHOPPER BAG	12.50
6 rows			

Explanation: Through examining the product sales per country, the retail store's owners can better understand the demand distribution. Therefore, they can optimize inventory and shipping operations to ensure these products are readily available and delivered on time.

Hide

```
# Find the highest-selling product in each country
highest_product_by_country <- country_product_sales %>%
  group_by(Country) %>%
  slice_max(
    order_by = Total_Sales,
    n = 1,
    with_ties = FALSE
  ) %>%
  ungroup()

# Display the results
highest_product_by_country
```

Country <chr>	StockCode <chr>	Description <chr>	Total_Sales <dbl>
Australia	48138	DOORMAT UNION FLAG	266.70
Austria	15056P	EDWARDIAN PARASOL PINK	142.80
Bahrain	22649	STRAWBERRY FAIRY CAKE TEAPOT	39.60
Belgium	22326	ROUND SNACK BOXES SET OF4 WOODLAND	743.40
Brazil	22366	DOORMAT AIRMAIL	67.50
Canada	21558	SKULL LUNCH BOX WITH CUTLERY	45.90
Channel Islands	21621	VINTAGE UNION JACK BUNTING	187.00
Cyprus	82484	WOOD BLACK BOARD ANT WHITE FINISH	146.10
Czech Republic	21428	SET3 BOOK BOX GREEN GINGHAM FLOWER	51.00
Denmark	23256	CHILDRENS CUTLERY SPACEBOY	149.40
1-10 of 38 rows			Previous 1 2 3 4 Next

Explanation: The highest-selling product per country provides the retail store's owners with insight into which product contributes most to its revenue. Its owners may then employ additional advertising strategies (eg. social media/tv ads) to drive more sales of these products.

Hide

```
# Calculate quantity sold for each product within each country
country_product_quantity <- df_clean %>%
  group_by(Country, StockCode, Description) %>%
  summarise(
    Total_Quantity = sum(Quantity, na.rm = TRUE),
    .groups = "drop"
  )

# Find the product with the highest quantity sold in each country
highest_quantity_by_country <- country_product_quantity %>%
  group_by(Country) %>%
  slice_max(
    order_by = Total_Quantity,
    n = 1,
    with_ties = FALSE
  ) %>%
  ungroup()

highest_quantity_by_country
```

Country <chr>	StockCode <chr>	Description <chr>	Total_Quantity <dbl>
Australia	22087	PAPER BUNTING WHITE LACE	48
Austria	22326	ROUND SNACK BOXES SET OF4 WOODLAND	36
Bahrain	85040A	S/4 PINK FLOWER CANDLES IN BOWL	12
Belgium	22326	ROUND SNACK BOXES SET OF4 WOODLAND	252
Brazil	21166	COOK WITH WINE METAL SIGN	12
Canada	23190	BUNDLE OF 3 SCHOOL EXERCISE BOOKS	24
Channel Islands	20725	LUNCH BAG RED RETROSPOT	70
Cyprus	21974	SET OF 36 PAISLEY FLOWER DOILIES	24
Czech Republic	22930	BAKING MOULD HEART MILK CHOCOLATE	18

Country <chr>	StockCode <chr>	Description <chr>	Total_Quantity <dbl>
Denmark	22629	SPACEBOY LUNCH BOX	72
1-10 of 38 rows			Previous 1 2 3 4 Next

Explanation: The highest-selling product per country and product with highest quantity sold per country differ. Yet, the retail store's owners can still capitalize on maintaining adequate inventory levels as these products still contribute to revenue though to a smaller degree.

Hide

```
# Calculate the number of orders by month
monthly_orders <- df_clean %>%
  group_by(YearMonth) %>%
  summarise(
    Number_of_Orders = length(
      unique(InvoiceNo)
    )
  )

head(monthly_orders)
```

YearMonth <chr>	Number_of_Orders <int>
2010-12	1365
2011-01	965
2011-02	972
2011-03	1296
2011-04	1087
2011-05	1466
6 rows	

Explanation: The number of orders per month aids the retail store's owners in differentiating between more customers or more orders versus larger orders from existing customers.

[Hide](#)

```
# Compare monthly revenue with monthly orders
monthly_summary <- merge(
  monthly_sales,
  monthly_orders,
  by = "YearMonth"
)

head(monthly_summary)
```

	YearMonth <chr>	Total_Sales <dbl>	Number_of_Orders <int>
1	2010-12	339693.4	1365
2	2011-01	266619.6	965
3	2011-02	231309.4	972
4	2011-03	295317.4	1296
5	2011-04	234846.6	1087
6	2011-05	326801.7	1466
6 rows			

Explanation: The retail store's owners can get a better understanding of monthly sales performance through a side-by-side view of sales and number of orders. From a glance at this data, it appears a few orders generate much revenue per month.

[Hide](#)

```
# Calculate average order value
monthly_summary$Average_Order_Value <-
  monthly_summary$Total_Sales /
  monthly_summary$Number_of_Orders

head(monthly_summary)
```

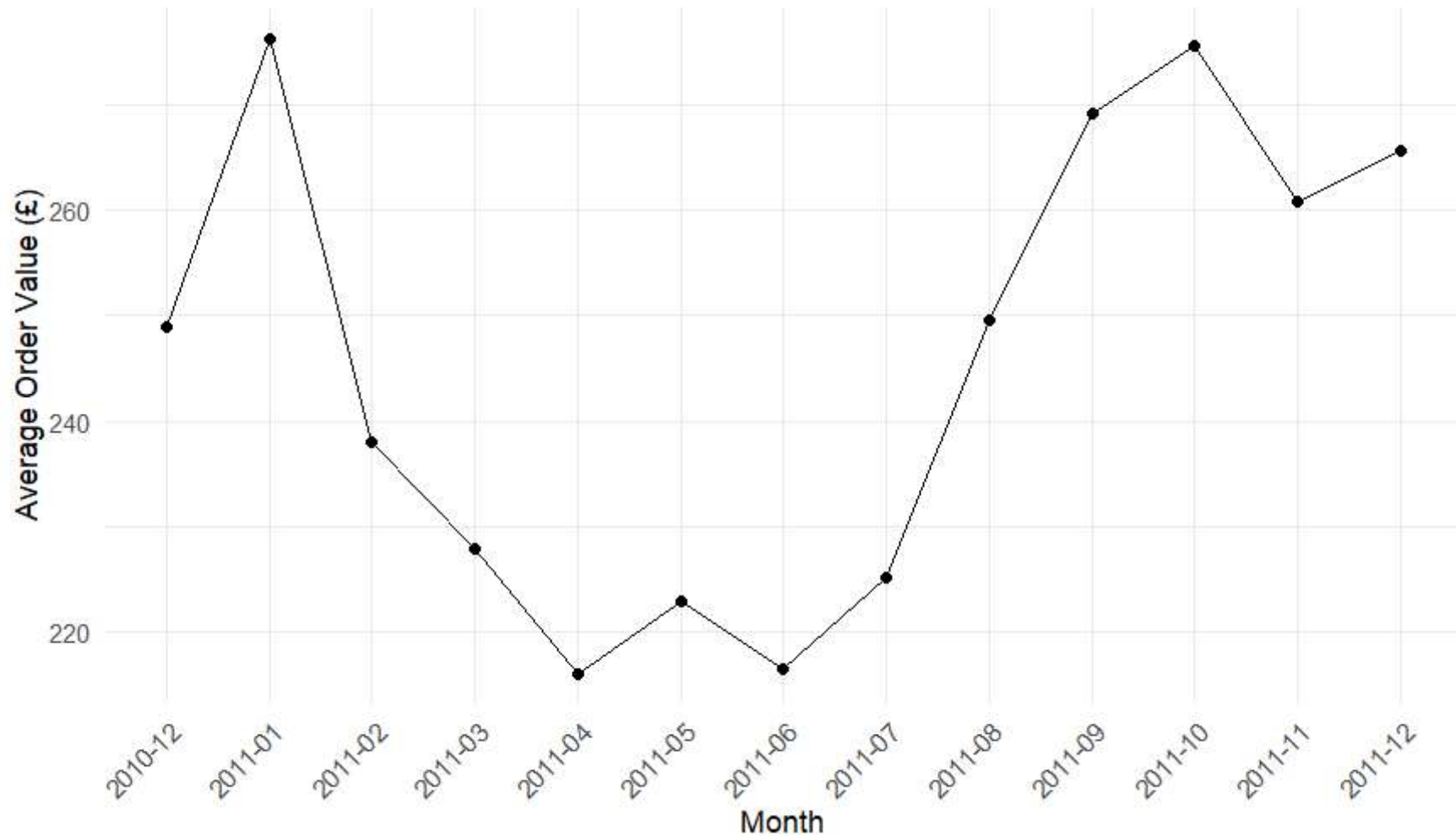
	YearMonth <chr>	Total_Sales <dbl>	Number_of_Orders <int>	Average_Order_Value <dbl>
1	2010-12	339693.4	1365	248.8597
2	2011-01	266619.6	965	276.2897
3	2011-02	231309.4	972	237.9726
4	2011-03	295317.4	1296	227.8683
5	2011-04	234846.6	1087	216.0503
6	2011-05	326801.7	1466	222.9207
6 rows				

Explanation: The data shows that based on the first six results, that each order generated on average £300 (rounded). In other words, this is the average spend per customer on an order. As the number of orders and average order value fluctuate, revenue is driven by both more orders and greater customer spend and not just one factor.

Hide

```
# Plot average order value
ggplot(
  monthly_summary,
  aes(
    x = YearMonth,
    y = Average_Order_Value,
    group = 1
  )
) +
  geom_line() +
  geom_point() +
  labs(
    title = "Monthly Average Order Value",
    x = "Month",
    y = "Average Order Value (£)"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
)
```

Monthly Average Order Value



Explanation: The data shows that customers spent varying amounts on orders over 2010-2011. The greatest spend were in Jan 2011 and Oct 2011, with a drastic decrease from Feb 2011 to Jul 2011. The low points were prime opportunities for the store's owners to implement product bundling, cross-selling/upselling, discounts and other promotions to improve sales.

Hide

```
# Calculate customer-level sales measures
customer_value <- df_clean %>%
  filter(!is.na(CustomerID)) %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Sales = sum(Sales, na.rm = TRUE),
    Number_of_Orders = n_distinct(InvoiceNo),
    Total_Quantity = sum(Quantity, na.rm = TRUE),
    Average_Order_Value = Total_Sales / Number_of_Orders,
    .groups = "drop"
  )

# Find the customer with the highest total sales
most_valuable_customer <- customer_value %>%
  slice_max(
    order_by = Total_Sales,
    n = 1,
    with_ties = FALSE
  )

most_valuable_customer
```

CustomerID <dbl>	Total_Sales <dbl>	Number_of_Orders <int>	Total_Quantity <dbl>	Average_Order_Value <dbl>
14911	71856.09	191	26425	376.2099
1 row				

Explanation: Customer with ID# 14911 was identified as the “most valuable customer” during the period and therefore could be extended special offers like free shipping once per month or coupons to encourage their continued patronage to the retail store.

Hide

```
# Find the top 10 customers by total sales
top_10_customers <- customer_value %>%
  arrange(desc(Total_Sales)) %>%
  head(10)

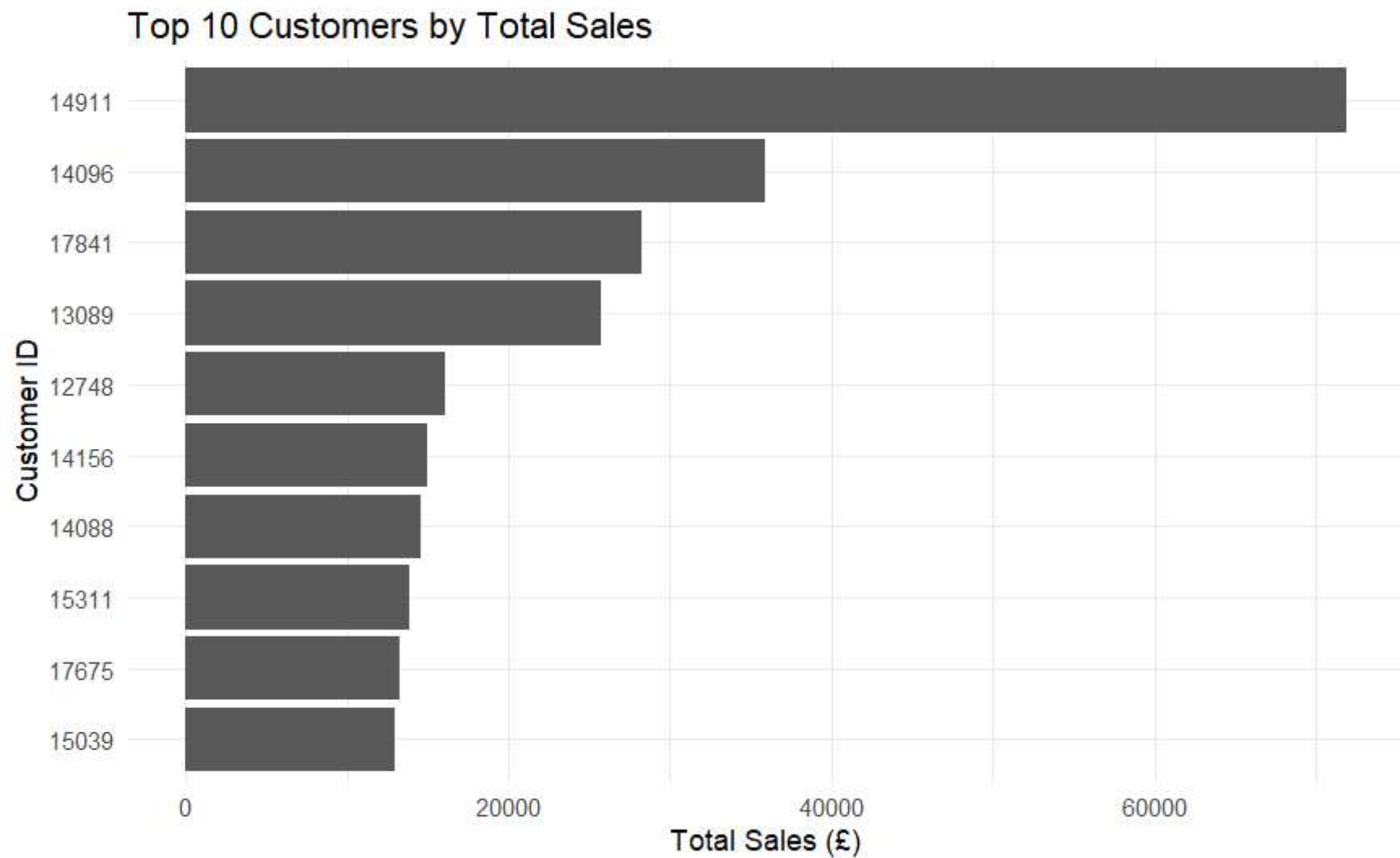
# Display the results
top_10_customers
```

CustomerID <dbl>	Total_Sales <dbl>	Number_of_Orders <int>	Total_Quantity <dbl>	Average_Order_Value <dbl>
14911	71856.09	191	26425	376.20990
14096	35914.24	17	11093	2112.60235
17841	28270.68	123	11667	229.84293
13089	25778.43	81	9199	318.25222
12748	16038.01	181	6497	88.60779
14156	14941.55	37	4758	403.82568
14088	14604.34	13	2844	1123.41077
15311	13889.20	91	4889	152.62857
17675	13274.21	31	4466	428.20032
15039	12997.87	47	4927	276.55043
1-10 of 10 rows				

Explanation: These customers constitute the retail store’s “high-revenue customer group”. Likewise, they could be extended special offers to maintain their loyalty.

Hide

```
# Plot top 10 customers by total sales
ggplot(
  top_10_customers,
  aes(
    x = reorder(
      as.factor(CustomerID),
      Total_Sales
    ),
    y = Total_Sales
  )
) +
  geom_bar(
    stat = "identity"
  ) +
  coord_flip() +
  labs(
    title = "Top 10 Customers by Total Sales",
    x = "Customer ID",
    y = "Total Sales (£)"
  ) +
  theme_minimal()
```



Explanation: The high revenue experienced by the online retail store can be credited to Customer with ID# 14911, as this customer's sales far exceed the others in the high-revenue customer group. Data on this customer's purchasing frequency would further substantiate this finding.

Hide

```
# Useful final management summary
total_revenue <- sum(
  df_clean$Sales,
  na.rm = TRUE
)

total_orders <- length(
  unique(df_clean$InvoiceNo)
)

total_products_sold <- sum(
  df_clean$Quantity,
  na.rm = TRUE
)

average_order_value <-
  total_revenue / total_orders

cat(
  "Total Revenue: £",
  round(total_revenue, 2),
  "\n"
)
```

Total Revenue: £ 4341420

Hide

```
cat(
  "Total Orders:",
  total_orders,
  "\n"
)
```

Total Orders: 17590

Hide

```
cat(  
  "Total Products Sold:",  
  total_products_sold,  
  "\n"  
)
```

Total Products Sold: 1580945

Hide

```
cat(  
  "Average Order Value: £",  
  round(average_order_value, 2),  
  "\n"  
)
```

Average Order Value: £ 246.81

Explanation: Over the course of 2010-2011, the online retail store earned £4,341,420 in total revenue from 17590 orders of 1,580,945 products valued at £246.81 on average per order.